



ResearchMind AI

Autonomous Academic Paper Deconstruction, Senior Peer Review, Grounded Chat & Cross-Paper Synthesis Agent

PLATFORM VERSION

2.0.0 (Production)

AI CORE MODELS

Gemini 3.5 / 2.5 Flash & Pro

REPOSITORY

github.com/darursrikar-sketch/researchmind-ai

DOCUMENT DATE

September 2026

1. Executive Summary & Problem Statement

Modern scientific research is experiencing exponential velocity. In machine learning, computer science, and biomedicine alone, thousands of preprints are submitted to arXiv, bioRxiv, and peer-reviewed venues every week. Researchers, engineers, graduate students, and technical leaders face severe bottlenecks:

- **Time Constraints:** Thoroughly digesting a 15–30 page dense research paper with mathematical appendices takes 2–4 hours.
- **Surface-Level LLM Summaries:** Standard consumer chatbots output generic bullet points that miss crucial nuances, fail to evaluate methodological flaws, ignore missing baselines, and hallucinate unsupported assertions.
- **Neglect of Mathematical Formulations:** Conventional summarizers skip LaTeX equations, loss functions, and architectural bounds entirely.
- **Fragmented Workflows:** Researchers juggle multiple disconnected tools for paper retrieval, PDF annotation, chat interaction, and literature synthesis.

The ResearchMind Solution

ResearchMind AI is an autonomous, academic-grade AI research partner designed to bridge the gap between rapid literature scanning and rigorous peer review. Powered by Google Gemini (`gemin-2.5-flash` , `gemin-3.5-flash` , and `gemin-2.5-pro` via the official `google-genai` SDK), ResearchMind delivers multi-source ingestion, autonomous 6-pillar deep deconstruction, page-grounded conversational Q&A, and cross-paper comparative meta-analysis.

2. Core Capabilities & Value Proposition

Capability	Standard Consumer Chatbots	ResearchMind AI Platform
Paper Ingestion	Manual copy-paste or raw file text dumps	One-click arXiv ID/URL retrieval + Page-aware PDF parsing
Citation Precision	General references (prone to hallucination)	Strict page-grounded citations (e.g., [Page 4]) verified against document text
Math & Equations	Plain text mangling of Greek symbols	Native KaTeX LaTeX rendering ($\dots$ and $\dots$) for loss functions and bounds
Critical Rigor	Polite, uncritical summaries	Senior reviewer simulation (NeurIPS/ICML style, 1–10 score, rebuttal questions)
Replication Details	High-level conceptual overview	Hyperparameters, compute budgets, subtle failure modes, runnable pseudocode
Cross-Paper Analysis	Limited to single context window	Multi-paper comparative synthesis matrix with architectural trade-off trees
Streaming Delivery	Monolithic text chunks	Real-time Server-Sent Events (SSE) streaming per analytical module
Export Formats	Raw clipboard copy	Formatted GitHub Markdown, standalone print-ready HTML, structured JSON

3. System Architecture & Data Flow

ResearchMind AI is structured into four decoupled, modular layers ensuring seamless scalability, robust error isolation, and maximum extensibility:

```
+-----+ | 1. INGESTION LAYER
| | • arXiv API Client: Resolves Atom XML, downloads full-text PDF to local cache | | • PDF Extraction
Engine: Uses pypdf for page-indexed tokenization and parsing | +-----+
+-----+ | ▼ +-----+
+-----+ | 2. AGENT CORE (Google Gemini via official google-genai SDK) | | •
PaperAnalysisAgent: Orchestrates 6 structured analytical modules | | • PaperChatEngine: Grounded
conversational Q&A enforcing exact page citations | | • PaperComparisonAgent: Cross-paper comparative
synthesis & trade-off matrix | +-----+
+-----+ | ▼ +-----+ | 3.
PERSISTENCE & STORAGE LAYER | | • SQLite (data/researchmind.db): Multi-tenant papers, analyses, users,
schemas | | • Upload Storage (data/uploads/): Retains original scientific PDFs | | • Security: PBKDF2/SHA-
256 salted password hashing & isolated user workspaces | +-----+
+-----+ | ▼ +-----+
+-----+ | 4. PRESENTATION & CLIENT INTERFACES | | • NexaCore Web Dashboard: Tailwind CSS,
Floating Island Navbar, SSE, KaTeX | | • Headless CLI: run_cli.py for batch scripts, REPL chat, and direct
exports | | • Exporters: Markdown (.md), Standalone HTML (.html), and JSON (.json) | +-----+
+-----+
```

3.1 Component Directory Structure

```

research mind project/
├── .env                # Environment configuration & API credentials
├── requirements.txt    # Python package dependencies
├── run_web.py          # Flask web server entrypoint (Port 5000)
├── run_cli.py          # Rich terminal CLI entrypoint
├── sample_paper.pdf    # Pre-packaged sample scientific paper
├── sample_papers/     # Sample benchmark library (Attention Is All You Need, etc.)
├── data/              # Persistent data folder
│   ├── researchmind.db # Embedded SQLite database
│   └── uploads/        # Uploaded and fetched PDF documents
└── researchmind/      # Primary application package
    ├── config.py       # Environment settings & Gemini Client factory
    ├── paper_loader.py # PDF extraction & arXiv API fetcher
    ├── storage.py       # SQLite persistence, schema migrations & auth
    ├── exporter.py      # Markdown, HTML, and JSON document serializers
    ├── cli.py           # CLI command handlers & interactive REPL
    ├── core/
    │   ├── prompts.py  # Domain-specific academic prompt templates
    │   ├── agent.py     # Multi-module PaperAnalysisAgent
    │   ├── chat_engine.py # Grounded Q&A engine with citations
    │   └── comparative.py # Multi-paper cross-synthesis agent
    └── web/
        ├── app.py       # Flask REST API & SSE streaming endpoints
        ├── templates/index.html # NexaCore-inspired reactive SPA UI
        └── static/
            ├── css/custom.css # Design tokens, hover glows & animations
            └── js/app.js      # Client-side state, streaming parser & KaTeX hooks

```

4. The 6-Pillar Autonomous Analysis Pipeline

When an ingested paper is submitted for analysis, ResearchMind triggers an automated pipeline across six specialized analytical modules:

1. Executive Summary & TL;DR

Distills the core problem vector, novel paradigm contributions, and headline quantitative metrics into an immediate high-signal briefing.

2. Methodology & Mathematical Architecture

Step-by-step pipeline deconstruction, mathematical loss functions, tensor shapes, and theoretical proofs rendered in clean LaTeX.

3. Critical Peer Review

Simulates a senior tier-1 reviewer (NeurIPS/ICML) with a 1–10 score, accept/reject recommendation, missing baselines, and author rebuttal questions.

4. Findings & Empirical Benchmarks

Comprehensive catalog of datasets, SotA performance comparison tables, statistical significance tests, and ablation study insights.

5. Limitations & Theoretical Bounds

Identifies fundamental failure modes, asymptotic scaling bottlenecks, memory ceilings, and unaddressed safety/adversarial concerns.

6. Implementation & Replication Blueprint

Actionable ML engineering guide featuring hyperparameter tables, GPU compute budgets, unstated implementation pitfalls, and Python pseudocode.

5. Interactive Grounded Chat & Page Citations

General conversational LLMs frequently invent facts when answering technical inquiries. The

PaperChatEngine eliminates this through strict architectural grounding:

- **Page-Indexed Context:** The ingested paper is partitioned into page-tagged buffers. Prompts explicitly constrain responses to cited evidence.
- **Verifiable Citations:** Every factual claim is accompanied by a page anchor (e.g., **[Page 4]**). The frontend renders these with distinctive badges.
- **Automated Starter Questions:** Upon paper ingestion, the agent analyzes the abstract to synthesize four targeted, domain-specific exploratory questions.
- **Stateful History:** Multi-turn discussions retain full context for follow-up questions while protecting against context-window saturation.

6. Cross-Paper Comparative Synthesis Matrix

The **PaperComparisonAgent** empowers researchers to select two or more papers from their local library to generate a side-by-side comparative meta-analysis:

1. **Comparative Matrix Table:** Direct juxtaposition of model architectures, objective functions, parameter scales, and benchmark scores.
2. **Synergies & Methodological Divergences:** Identifies where papers align on foundational principles and where empirical conclusions conflict.
3. **Architectural Decision Tree:** Clear guidelines for practitioners indicating when to select Architecture A vs. Architecture B based on inference latency, training budget, and dataset constraints.

7. Storage & Database Schema

ResearchMind stores data in an embedded SQLite database (**data/researchmind.db**) with automatic schema migration and multi-user isolation:

Table	Primary Key	Key Columns	Description
users	id (TEXT)	username , email , password_hash , created_at	User credentials for multi-tenant workspace separation. Passwords hashed using PBKDF2/SHA-256.
papers	id (TEXT)	title , authors , abstract , arxiv_id , pdf_path , num_pages , data_json , user_id	Ingested scientific papers, metadata, file pointers, and serialized page text mappings.
analyses	paper_id (TEXT)	paper_title , model_used , data_json , timestamp , user_id	Cached analysis outputs across the 6 pillars, preserving execution times and model identifiers.

8. REST & Server-Sent Events (SSE) API Specification

Method	Route	Type	Description
GET	/api/auth/me	JSON	Returns active user session details or null
POST	/api/auth/signup	JSON	Registers a new user account
POST	/api/auth/signin	JSON	Authenticates credentials and establishes session cookie
POST	/api/auth/signout	JSON	Invalidates active session
GET / POST	/api/config	JSON	Reads or updates Gemini API key and default model
GET	/api/papers	JSON	Lists all papers belonging to the active workspace
GET	/api/papers/<id>	JSON	Retrieves paper metadata and cached analysis results
DELETE	/api/papers/<id>	JSON	Deletes paper and associated analyses from disk & database
POST	/api/upload	JSON	Uploads and parses a multipart PDF document
POST	/api/arxiv	JSON	Fetches paper from arXiv by ID or URL
POST	/api/analyze	SSE Stream	Streams real-time analysis across selected pillars
POST	/api/chat	SSE Stream	Streams grounded conversational answers with citations
GET	/api/chat/starters/<id>	JSON	Generates four tailored exploratory questions
POST	/api/compare	SSE Stream	Streams comparative cross-paper synthesis matrix
GET	/api/export/<id>? format=md html json	File / Text	Downloads analysis formatted as Markdown, HTML, or JSON

9. User Interface & NexaCore Design System

The web interface incorporates NexaCore's modern design language:

- **Floating Island Navbar** (`.nexa-nav-island`): Fixed top navigation with blurred backdrop, status indicator, model selector, and gradient action buttons.
- **Dynamic Lighting Sweeps** (`.nexa-glow-sweep`): Interactive cards feature smooth downward radial lighting sweeps on cursor hover.
- **Depth Physics** (`.nexa-bottom-shade`): Card bases fade smoothly into dark glassmorphic shades when active.
- **Sliding Action Triggers** (`.nexa-reveal-btn`): Action buttons smoothly expand from zero height on hover with vibrant gradient fills.
- **Structured Delivery Timeline**: Four connected workflow milestones linked by a multi-stop vertical gradient line (`.nexa-grad-line-bg`).

10. CLI & Automated Scripting Interface

For automated batch pipelines or headless Linux environments, `run_cli.py` provides a full terminal suite:

```
# 1. Analyze an arXiv paper and stream findings to terminal
python run_cli.py analyze 1706.03762

# 2. Analyze a local PDF and export directly to Markdown
python run_cli.py analyze ./sample_paper.pdf --export md --out ./report.md

# 3. Analyze using Gemini 2.5 Pro for deep critical review
python run_cli.py analyze 2106.09685 --model gemini-2.5-pro --export html

# 4. Start an interactive terminal REPL chat with grounded citations
python run_cli.py chat 1706.03762

# 5. Synthesize a cross-paper comparison
python run_cli.py compare 1706.03762 2106.09685 --out comparison.md

# 6. List papers in local repository
python run_cli.py list

# 7. Configure Gemini credentials
python run_cli.py config --set-key AIzaSy... --set-model gemini-3.5-flash
```

11. Installation, Configuration & Operations

1. Clone Repository:

```
git clone https://github.com/darursrikar-sketch/researchmind-ai.git
cd researchmind-ai
```

2. Install Dependencies:

```
pip install -r requirements.txt
```

3. Configure Credentials:

```
# In .env file
GEMINI_API_KEY=AIzaSy...
GEMINI_MODEL=gemini-3.5-flash
FLASK_SECRET_KEY=secure-random-key
```

4. Launch Application:

```
python run_web.py
```

Navigate to <http://127.0.0.1:5000> .

12. Verification & Test Suite

Run the automated test suite to ensure system integrity:

```
python -m unittest discover tests
```